माधव प्रौद्योगिकी एवं विज्ञान संस्थान, ग्वालियर (म.प्र.), भारत
MADHAV INSTITUTE OF TECHNOLOGY & SCIENCE, GWALIOR (M.P.), INDIA
Deemed University
(Declared under Distinct Category by Ministry of Education, Government of India)
NAAC ACCREDITED WITH A++ GRADE

Lab Report
of
IOT Lab (24243106)



SUBMITTED BY

YOGRAJ SINGH

GURJAR (BTIR24O1078)

V semester

Information Technology (Artificial Intelligence and Robotics)

SUBMITTED TO

Dr. Neeraj Mishra

Dr. Pawan Dubey

CENTRE FOR ARTIFICIAL INTELLIGENCE

Madhav Institute of Technology & Science

Gwalior - 474005 (MP) est. 1957

Session: 2026-27

INDEX

S.No.	Name of the Experiment	Date	Page No.	Remarks / Sign
1	Introduction to Arduino Board and Arduino IDE Installation and Setup	08/08/2026	3-6	
2	Program to Blink and Fade an LED Using Digital and Analog Output	08/08/2026	7-8	
3	Program to Measure Distance Using Ultrasonic Sensor	05/09/2026	8-11	
4	Study of Push Button Module Using Arduino	05/09/2026	12-15	
5	Study of LoRaWAN Communication Using SX1278 Module	05/09/2026	16-20	
6	Study of Bluetooth Module Using Arduino	12/09/2026	21-23	
7	Study of Air Quality Sensor Using Arduino	12/09/2026	24-27	
8	Study of RFID-Based Identification System Using Arduino	12/09/2026	28-30	
9	Study of Temperature and Humidity Measurement Using DHT22 Sensor	19/09/26	31-32	
10	To Interface GPS Module with Arduino and Display Location	19/09/26	33-35	

Experiment 1:

Introduction to Arduino Board and Arduino IDE Installation and Setup

Aim: To study the Arduino board and install and set up the Arduino IDE for programming.

Apparatus Required:

- Arduino Uno board
- USB cable
- Computer/Laptop
- Arduino IDE software

Theory:

Arduino is an open-source electronics platform used to develop embedded systems and IoT projects. It consists of a programmable microcontroller that can interact with sensors, actuators and other electronic components.

Arduino Uno is one of the most commonly used Arduino boards. It is based on the ATmega328P microcontroller and provides digital and analog input/output pins.

Main Components of Arduino Uno:

1. Microcontroller – Executes the uploaded program.
2. Digital I/O Pins – Used to read digital inputs and control outputs.
3. Analog Input Pins – Used to read analog signals from sensors.
4. USB Port – Used for programming and power supply.
5. Power Jack – Provides external power to the board.
6. Reset Button – Restarts the running program.
7. Power LED – Indicates that the board is powered on.

Arduino IDE:

Arduino IDE (Integrated Development Environment) is software used to write, compile and upload programs to Arduino boards. Arduino programs are called sketches.

Procedure:

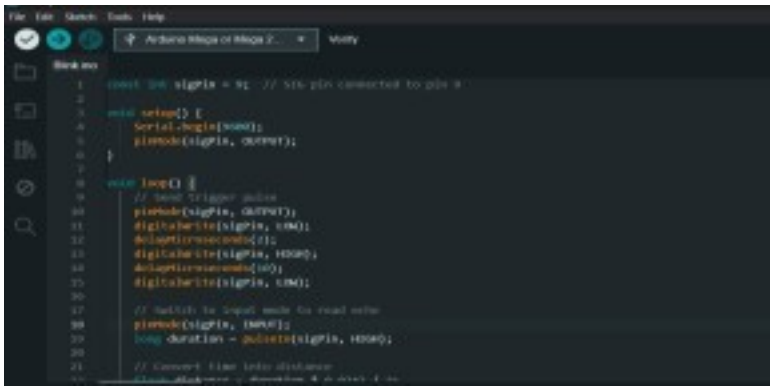
1. Download the Arduino IDE from the official Arduino website.



2. Install the software by following the installation instructions. 3. Connect the Arduino Uno board to the computer using a USB cable.

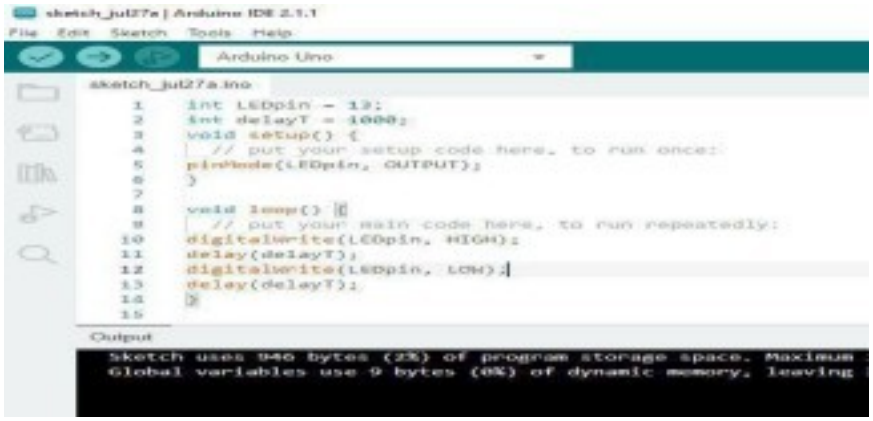


4. Open the Arduino IDE.
5. Select the appropriate board by navigating to Tools → Board → Arduino Uno.
6. Select the correct port from Tools → Port.



7. Write a simple program in the IDE.
8. Click the Verify button to compile the program.
9. Click the Upload button to upload the program to the Arduino board.
10. Check the output to confirm that the board and IDE are

working properly.



Sample Program:

```
void setup() {
 pinMode(LED_BUILTIN, OUTPUT);
}
void loop() {
 digitalWrite(LED_BUILTIN, HIGH);
 delay(1000);
 digitalWrite(LED_BUILTIN, LOW);
 delay(1000);
}
```

Output:

The built-in LED of the Arduino Uno blinks at an interval of one second, indicating that the Arduino board and IDE have been successfully set up.

Result:

The Arduino Uno board was studied, and the Arduino IDE was successfully installed and configured for programming.

Experiment-02

Program to Blink and Fade an LED Using Digital and Analog

Output Aim

To write an Arduino program to blink an LED using digital output and fade an LED using analog output (PWM).

Apparatus Required

- Arduino Uno
- LED
- 220Ω resistor
- Breadboard
- Jumper wires
- USB cable

Theory

Digital output has two states: HIGH (5V) and LOW (0V). It is used to turn the LED ON and OFF.

Analog output on Arduino Uno is achieved using PWM (Pulse Width Modulation). The analogWrite() function generates PWM signals to control the LED brightness.

Circuit Connections

LED Pin Arduino Pin

Anode (+) Pin 9 through 220Ω resistor

Cathode (−) GND

Program

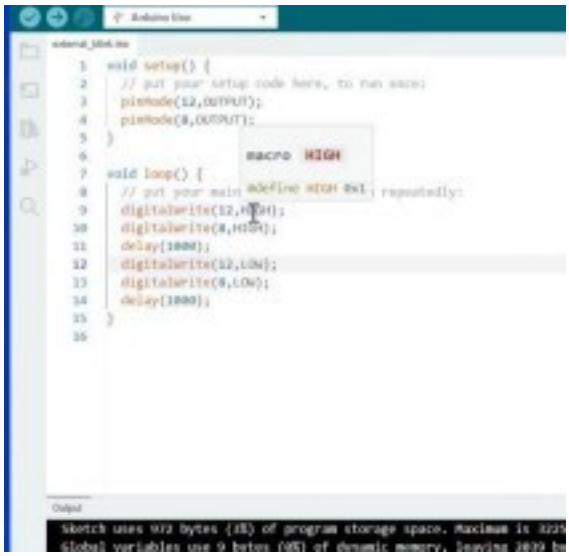
```
int led = 9;
void setup() {
 pinMode(led, OUTPUT);
}
void loop() {
 // Blink LED using digital output
 digitalWrite(led, HIGH);
 delay(1000);
```

```
digitalWrite(led, LOW);
delay(1000);
// Fade LED using PWM analog output
for (int brightness = 0; brightness <= 255; brightness++)
{ analogWrite(led, brightness);
delay(10);
}
for (int brightness = 255; brightness >= 0; brightness--)
{ analogWrite(led, brightness);
delay(10);
}
}
```

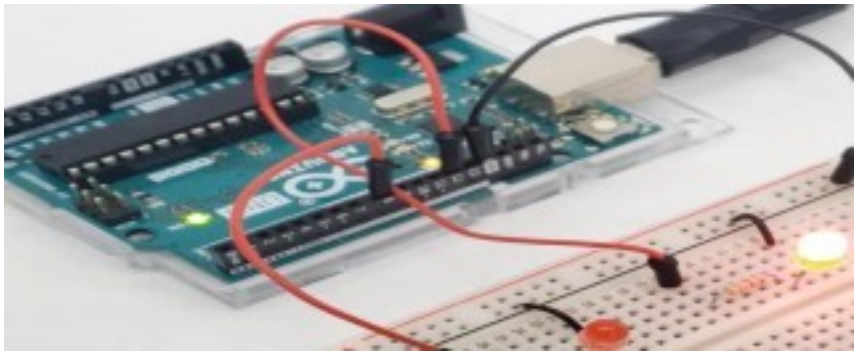
Procedure

1. Connect the LED to Arduino pin 9 through a 220Ω resistor.
2. Connect the LED's cathode to GND.
3. Connect Arduino to the computer using a USB cable. 4.

   Open Arduino IDE and select the correct board and COM port.
5. Upload the program.



6. Observe the LED blinking and gradually increasing and decreasing in brightness.



Output

The LED turns ON and OFF with a one-second delay, then gradually fades from dim to bright and bright to dim.

Result

The LED was successfully blinked using digital output and faded using PWM analog output.

Precautions

1. Always use a resistor in series with the LED.
2. Connect the LED with correct polarity.
3. Use a PWM-supported pin, such as pin 9 on Arduino Uno.

Experiment-03

Program to Measure Distance Using Ultrasonic Sensor

Aim

To measure the distance of an object using an ultrasonic sensor and Arduino Uno.

Apparatus Required

- Arduino Uno
- Ultrasonic Sensor (HC-SR04)
- Breadboard
- Jumper wires
- USB cable

Theory

The ultrasonic sensor measures distance by transmitting ultrasonic sound waves and receiving the reflected waves from an object.

The Arduino calculates the distance using the time taken by the sound wave to travel to the object and return.

Formula:

$$\text{Distance} = \frac{\text{Time} \times 0.0343}{2}$$

Here, 0.0343 cm/µs is the approximate speed of sound. We divide by 2 because the sound travels to the object and back.

Circuit Connections

HC-SR04 Pin Arduino Uno

VCC 5V

GND GND

TRIG Pin

9

ECHO Pin 10

Program

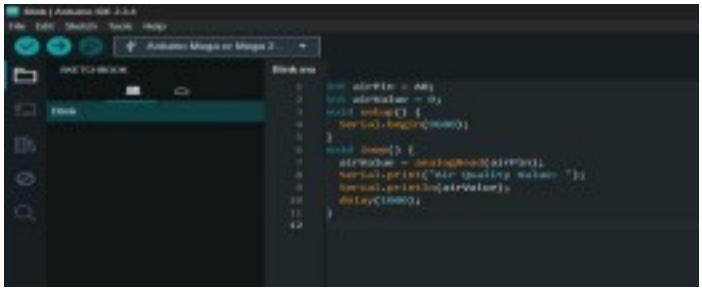
```
#define trigPin 9

#define echoPin 10
```

```
long
duration;
float
distance; void
setup() {
 pinMode(trigPin,
OUTPUT);
pinMode(echoPin, INPUT);
Serial.begin(9600);
}
void loop()
{ digitalWrite(trigPin,
LOW);
delayMicroseconds(2);
 digitalWrite(trigPin, HIGH);
delayMicroseconds(10);
digitalWrite(trigPin, LOW);
duration = pulseIn(echoPin,
HIGH); distance = duration *
0.0343 / 2; Serial.print("Distance:
");
 Serial.print(distance);
 Serial.println("  cm");
 delay(500);
}
```

Procedure

1. Connect the ultrasonic sensor to Arduino according to the circuit connections.



2. Connect Arduino to the computer using a USB cable.
3. Open Arduino IDE and select the correct board and COM port.

4. Upload the program to Arduino.

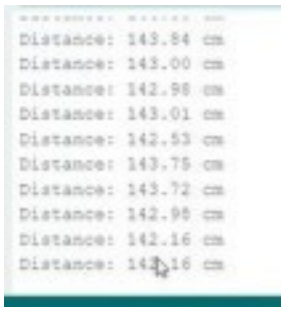
```
#include <LiquidCrystal.h>
LiquidCrystal lcd(1, 2, 4, 5, 6, 7);
const int trigPin = 9;
const int echoPin = 10;

long duration;
int distanceCm, distanceInch;

void setup() {
lcd.begin(16,2);
pinMode(trigPin, OUTPUT);
pinMode(echoPin, INPUT);
}

void loop() {
digitalWrite(trigPin, LOW);
delayMicroseconds(2);
```

5. Open the Serial Monitor and set the baud rate to 9600.
6. Place an object in front of the sensor and observe the measured distance.



Output

The distance of the object is displayed in centimeters on the Serial

Monitor. ## Result

The distance of an object was successfully measured using an ultrasonic sensor and Arduino Uno.

Precautions

1. Connect the sensor pins correctly.
2. Keep the object in front of the sensor for accurate measurement.
3. Avoid obstacles near the sensor that may cause incorrect readings

Experiment 4

Study of Push Button Module Using Arduino

Aim

To study the working of a Push Button Module and interface it with Arduino to control an LED.

Apparatus Required

1. Arduino UNO
2. Push Button Module
3. LED (built-in LED of Arduino)
4. Connecting Wires
5. USB Cable
6. Computer/Laptop

Theory

A push button is an input device used to provide a digital signal to a microcontroller. It works as a switch that changes its state when pressed.

The push button has two states:

• HIGH (1): The input pin receives a HIGH signal.

• LOW (0): The input pin receives a LOW signal.

When the push button is pressed, Arduino detects the input and turns ON the built-in LED. When the button is released, the LED turns OFF.

Push buttons are used in control panels, electronic devices and IoT systems to provide manual input.

Circuit Connections

Push Button Module Arduino UNO

VCC 5V

GND

GND

OUT / Signal Digital Pin 2

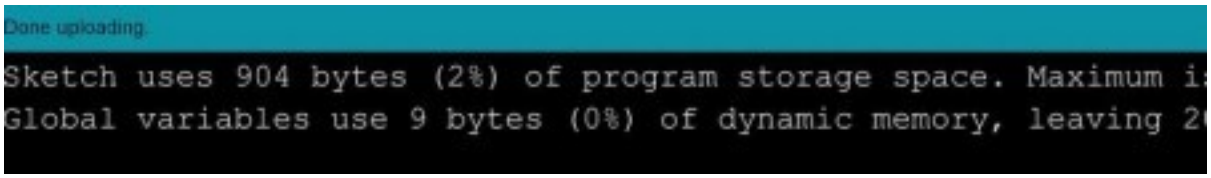
The module's exact pin labels may vary. Use the signal/output pin marked on your kit.

Procedure

1. Connect the VCC pin of the Push Button Module to the 5V pin of Arduino.



2. Connect the GND pin of the module to Arduino GND.
3. Connect the OUT pin of the module to digital pin 2.
4. Connect Arduino UNO to the computer using a USB cable.
5. Open Arduino IDE and select Arduino UNO from Tools → Board.
6. Select the correct port from Tools → Port.
7. Write and upload the following program.
8. Press the push button and observe the built-in LED.



9. Release the button and observe the LED again.

Arduino Program

```
int buttonPin = 2;
int ledPin = LED_BUILTIN;
void setup()
{ pinMode(buttonPin, INPUT);
pinMode(ledPin, OUTPUT);
}
void loop() {
 int buttonState = digitalRead(buttonPin);
 if (buttonState == HIGH)
 { digitalWrite(ledPin, HIGH);
 }
 else {
```

```
 digitalWrite(ledPin, LOW);
 }
}
```

Output

When the push button is pressed, the built-in LED of Arduino turns ON. When the button is released, the LED turns OFF.

Result

The working of the Push Button Module was studied and successfully interfaced with Arduino to control an LED.

Experiment-05

Study of LoRaWAN Communication Using SX1278

Module Aim

To study the working principle of LoRaWAN and understand long-range, low-power wireless communication using the LoRa SX1278 module.

Apparatus Required

1. Arduino UNO
2. LoRa SX1278 Module
3. Compatible LoRa Antenna
4. Connecting Wires
5. USB Cable
6. Computer/Laptop

Theory

LoRaWAN (Long Range Wide Area Network) is a wireless communication protocol designed for low-power, long-range communication between IoT devices.

LoRa is the physical-layer wireless modulation technology, while LoRaWAN is the network protocol that manages communication between end devices and gateways.

The SX1278 is a LoRa transceiver module that uses chirp spread spectrum modulation to transmit and receive data over long distances with low power consumption.

In a typical LoRaWAN architecture, end devices send data to gateways, which forward it to a network server for processing.

Applications:

- Smart agriculture
- Environmental monitoring
- Smart city applications
- Remote sensor monitoring
- Industrial IoT

Pin Description

Pin Function

VCC Power

supply

GND Ground

SCK SPI Clock

MISO SPI Data Output

MOSI SPI Data Input

NSS / CS Chip Select

RST Reset

DIO0 Interrupt / Status Signal

Circuit Connections

For an SX1278 module connected to Arduino UNO:

SX1278 Module Arduino UNO

VCC Compatible 3.3V supply

GND GND

SCK Pin 13

MISO Pin 12

MOSI Pin 11

NSS / CS Pin 10

RST Pin 9

DIO0 Pin 2

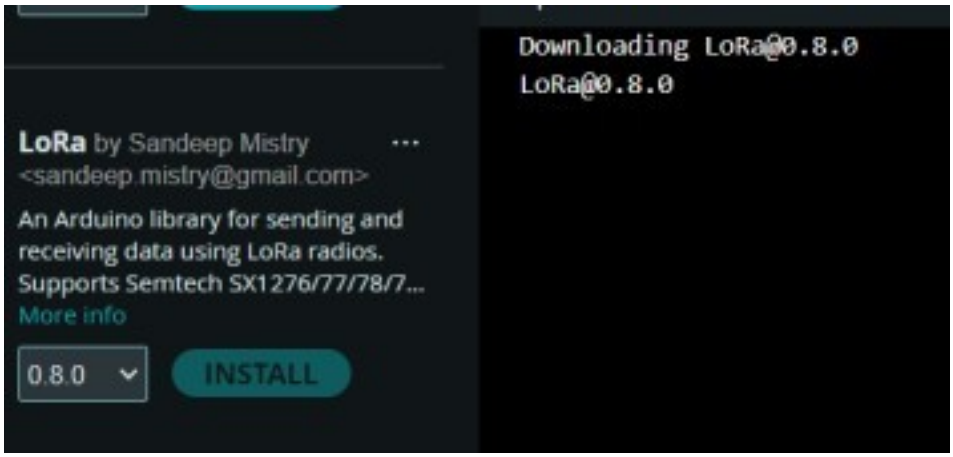
Important: The SX1278 is a 3.3V device. Check your kit's board and voltage requirements before connecting it to Arduino. Do not apply 5V directly to 3.3V-only pins. Keep a suitable antenna connected during radio transmission.

Procedure

1. Identify the LoRa SX1278 module and its pins.



2. Connect the module to Arduino UNO using the SPI interface.
3. Connect a compatible antenna to the module.
4. Connect Arduino to the computer using a USB cable.
5. Open Arduino IDE and select Arduino UNO and the correct port.
6. Install a suitable LoRa library, such as the LoRa library by Sandeep Mistry, if compatible with the module.



7. Configure the transmitter and receiver with matching radio settings and frequency permitted in the region.
8. Upload the transmitter program to one device and the receiver program to another compatible device.
9. Transmit a test message and observe the received data.
10. Record the communication results.

Arduino Program

This example demonstrates basic LoRa radio communication using the SX1278 with the Arduino LoRa library. It is LoRa point-to-point communication, not a complete LoRaWAN implementation.

Use two compatible LoRa modules for actual transmission and reception.

Transmitter Code

```
#include <SPI.h>

#include <LoRa.h>

void setup() {

Serial.begin(9600);

 if (!LoRa.begin(433E6))

 { Serial.println("LoRa initialization failed");
```

```
 while (1);
 }
 Serial.println("LoRa Transmitter Started");
}
void loop() {
 LoRa.beginPacket();
 LoRa.print("Hello from
 LoRa"); LoRa.endPacket();
 Serial.println("Message Sent");
 delay(2000);
}
```

Receiver Code

```
#include <SPI.h>
#include <LoRa.h>
void setup() {
Serial.begin(9600);
 if (!LoRa.begin(433E6))
 { Serial.println("LoRa initialization failed");
 while (1);
 }
 Serial.println("LoRa Receiver Started");
}
void loop() {
 int packetSize = LoRa.parsePacket();
 if (packetSize) {
 while (LoRa.available()) {
 Serial.print((char)LoRa.read());
 }
 Serial.println();
 }}
```

Output

The transmitter sends the message “Hello from LoRa” wirelessly. The compatible receiver displays the received message on the Serial Monitor.

Result

The working principle of LoRaWAN and the role of the SX1278 module in long-range, low power IoT communication were studied. Basic LoRa point-to-point data transmission was demonstrated.

Experiment-06

Study of Bluetooth Module Using Arduino

Aim

To study the working of a Bluetooth module and understand wireless data communication using Arduino.

Apparatus Required

1. Arduino UNO
2. Bluetooth Module (HC-05/HC-06)
3. Connecting Wires
4. USB Cable
5. Computer/Laptop

Theory

Bluetooth is a short-range wireless communication technology used to exchange data between electronic devices.

The HC-05 Bluetooth module provides wireless serial communication between Arduino and devices such as smartphones and computers.

It uses UART (Universal Asynchronous Receiver-Transmitter) for serial communication with Arduino.

Bluetooth is commonly used in home automation, wireless robotics and remote control applications.

Circuit Connections

Bluetooth Module Arduino UNO

VCC Compatible supply voltage

GND GND

TXD Digital Pin 10

RXD Digital Pin

11

Note: Check the module's voltage requirements. HC-05 breakout boards often accept 5V power, but the RX logic pin may require 3.3V-level signals.

Procedure

1. Connect the Bluetooth module to Arduino UNO.

2. Connect the module's TXD and RXD pins to Arduino's serial communication pins.
3. Connect Arduino to the computer using a USB cable.
4. Open Arduino IDE and select the correct board and port.
5. Install the SoftwareSerial library if required (it is included with Arduino AVR boards).
6. Upload the program given below.
7. Pair the Bluetooth module with a compatible smartphone or computer.
8. Send data wirelessly and observe the communication.

Arduino Program

```
#include
<SoftwareSerial.h>
SoftwareSerial BT(10, 11);
void setup() {
Serial.begin(9600);
BT.begin(9600);
 Serial.println("Bluetooth Ready");
}
void loop() {
 if (BT.available())
 { char data =
 BT.read();
 Serial.print(data);}
 if (Serial.available()) {
 BT.write(Serial.read());
 }}
```

Output

The Bluetooth module establishes wireless serial communication with a paired device. Data sent from the Bluetooth device is displayed on the Arduino Serial Monitor.

Result

The working of the Bluetooth module was studied, and wireless serial communication using Arduino was understood.

Experiment-07

Study of Air Quality Sensor Using Arduino

Aim

To study the working of an Air Quality Sensor and detect changes in air quality using Arduino.

Apparatus Required

1. Arduino UNO
2. Air Quality Sensor Module
3. Connecting Wires
4. USB Cable
5. Computer/Laptop

Theory

An air quality sensor is an electronic device used to detect changes in the quality of surrounding air. It can detect certain gases, smoke or air pollutants depending on the type of sensor used.

The sensor converts changes in the surrounding air into electrical signals. Arduino reads these signals and displays the sensor values on the Serial Monitor.

Air quality sensors are used in IoT applications for environmental monitoring, indoor air quality measurement and pollution detection.

Circuit Connections

Air Quality Sensor Arduino UNO

VCC 5V*

GND

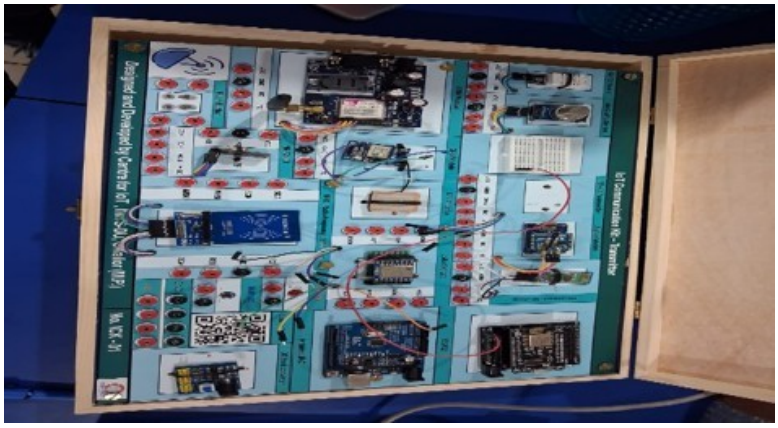
GND AO

A0

Check the operating voltage printed on your sensor module.

Procedure

1. Connect the VCC pin of the air quality sensor to the appropriate power supply.



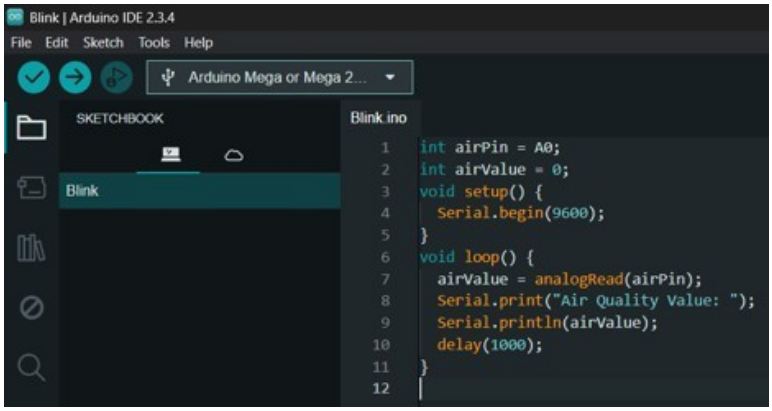
2. Connect the GND pin to Arduino GND.

3. Connect the analog output (AO) to Arduino analog pin A0.

4. Connect Arduino UNO to the computer using a USB cable.

5. Open Arduino IDE and select Arduino UNO and the correct port.

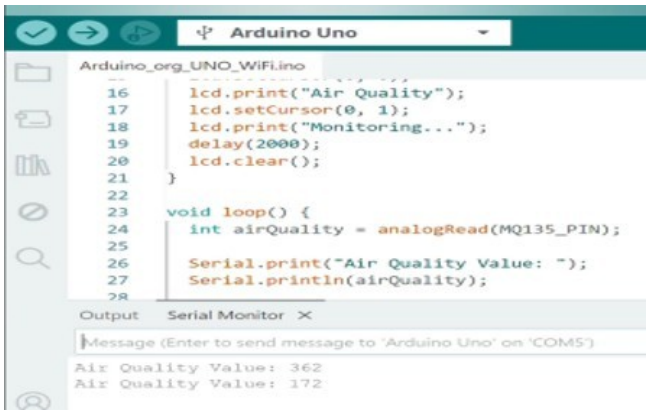
6. Upload the program given below.



7. Open the Serial Monitor and set the baud rate to 9600.

8. Observe the sensor readings under normal surrounding conditions.

9. Record any changes in the readings.



Arduino Program

```
int airPin = A0;

int airValue =

0; void setup()

{

Serial.begin(9600);

}

void loop() {

airValue = analogRead(airPin);

Serial.print("Air Quality Value: ");

Serial.println(airValue);

delay(1000);

}
```

Output

The air quality sensor readings are displayed on the Serial Monitor. The readings may change according to the surrounding air conditions.

Result

The working of the Air Quality Sensor was studied, and its application in air quality monitoring using Arduino was understood.

Experiment–08

Study of RFID-Based Identification System Using Arduino

Aim

To study the working of an RFID module and identify RFID cards or tags using Arduino.

Apparatus Required

1. Arduino UNO
2. RFID Reader Module (RC522)
3. RFID Card or Tag
4. Connecting Wires
5. USB Cable
6. Computer/Laptop

Theory

RFID (Radio Frequency Identification) is a wireless technology used to identify and track objects using radio waves.

An RFID system consists of an RFID reader and an RFID tag or card. The reader generates a radio-frequency field, and when a compatible tag is brought near it, the tag transmits its identification data to the reader.

The Arduino receives the data from the RFID reader and displays the card's unique identification number (UID) on the Serial Monitor.

RFID is commonly used in attendance systems, access control, inventory management and smart identification systems.

Pin Description

RFID RC522 $_{Pin}$Function

SDA / SS Chip Select

SCK Serial Clock

MOSI Data Input

MISO Data Output

IRQ Interrupt

GND
Ground RST
Reset

3.3V Power Supply

Circuit Connections

For an RC522 module connected to Arduino UNO:

RFID RC522 Arduino UNO

SDA / SS Pin 10

SCK Pin 13

MOSI Pin 11

MISO Pin 12

RST Pin 9

GND GND

3.3V 3.3V

Note: RC522 operates at 3.3V. Do not connect its power pin to 5V. Its signal pins are also 3.3V logic, so use appropriate level shifting where required.

Procedure

1. Identify the RFID reader module and RFID card/tag.
2. Connect the RC522 module to Arduino UNO according to the circuit connections.
3. Connect Arduino to the computer using a USB cable.
4. Open Arduino IDE and select Arduino UNO and the correct port.
5. Install the MFRC522 library using the Library Manager.

6. Upload the program given below.



```
RFID_ReadUID
//Arduino Code - RC522 Read RFID Tag UID

#include <SPI.h>
#include <MFRC522.h>

#define SS_PIN 10
#define RST_PIN 7

MFRC522 rfid(SS_PIN, RST_PIN); // Instance of the class

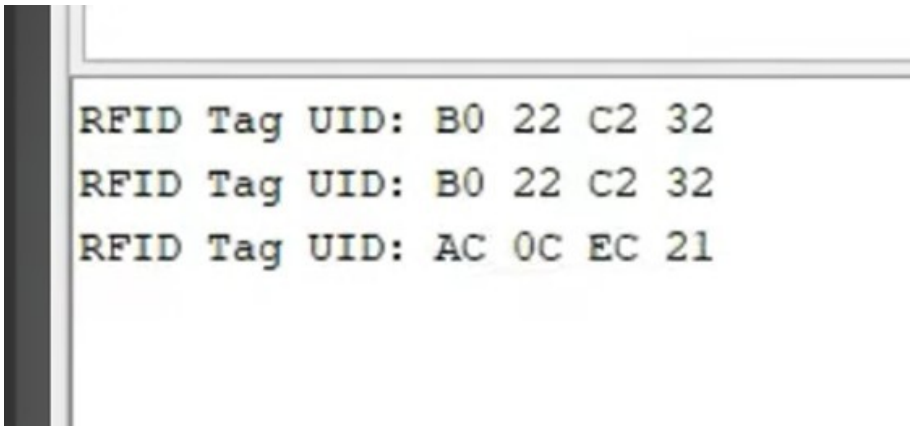
MFRC522::MIFARE_Key key;

void setup() {
  Serial.begin(9600);
  SPI.begin(); // Init SPI bus
  rfid.PCD_Init(); // Init MFRC522
}

void loop() {

  // Reset the loop if no new card present on the sensor/reader. This saves the entire process when idle.
  if ( ! rfid.PICC_IsNewCardPresent())
    return;
```

7. Open the Serial Monitor and set the baud rate to 9600.
8. Bring the RFID card or tag near the reader.
9. Observe the UID displayed on the Serial Monitor.



```
RFID Tag UID: B0 22 C2 32
RFID Tag UID: B0 22 C2 32
RFID Tag UID: AC 0C EC 21
```

10. Repeat the process using another compatible RFID card or tag.

Arduino Program

```
#include <SPI.h>
#include
<MFRC522.h> #define
SS_PIN 10
#define RST_PIN 9
MFRC522 rfid(SS_PIN, RST_PIN);
void setup() {
 Serial.begin(9600);
 SPI.begin();
 rfid.PCD_Init();
```

```
 Serial.println("RFID Reader Ready");
}
void loop() {
 if (!
 rfid.PICC_IsNewCardPresent())
 return;
 if (!
 rfid.PICC_ReadCardSerial())
 return;
 Serial.print("Card UID: ");
 for (byte i = 0; i < rfid.uid.size; i++) {
 Serial.print(rfid.uid.uidByte[i],
 HEX); Serial.print(" ");
 }
 Serial.println();
 rfid.PICC_HaltA();
 rfid.PCD_StopCrypto1();
}
```

Output

When an RFID card or tag is brought near the reader, its unique identification number (UID) is displayed on the Serial Monitor.

Result

The working of the RFID module was studied, and RFID card identification using Arduino was successfully demonstrated.

Precautions

1. Connect the RC522 module to the correct 3.3V supply.
2. Check all connections before powering the circuit.
3. Use a compatible RFID card or tag.
4. Keep the card close to the reader during scanning.
5. Avoid short circuits while connecting the module.

Experiment–09

Study of Temperature and Humidity Measurement Using DHT22

Sensor **Aim**

To measure temperature and relative humidity using the DHT22 sensor interfaced with Arduino UNO.

Apparatus Required

1. Arduino UNO
2. DHT22 Temperature and Humidity Sensor
3. Connecting Wires
4. USB Cable
5. Computer/Laptop

Theory

The DHT22 is a digital sensor used to measure the temperature and relative humidity of the surrounding environment.

It consists of a temperature sensing element and a capacitive humidity sensing element. The sensor processes the measured values and sends digital data to the Arduino.

Arduino reads the sensor data and displays the temperature in degrees Celsius and relative humidity in percentage on the Serial Monitor.

The DHT22 sensor is used in weather monitoring, smart homes, agriculture and IoT-based environmental monitoring systems.

Pin Description

Pin Function

VCC Power Supply

DATA Digital Data

Output NC Not Connected

GND Ground

Circuit Connections

DHT22 Sensor Arduino UNO

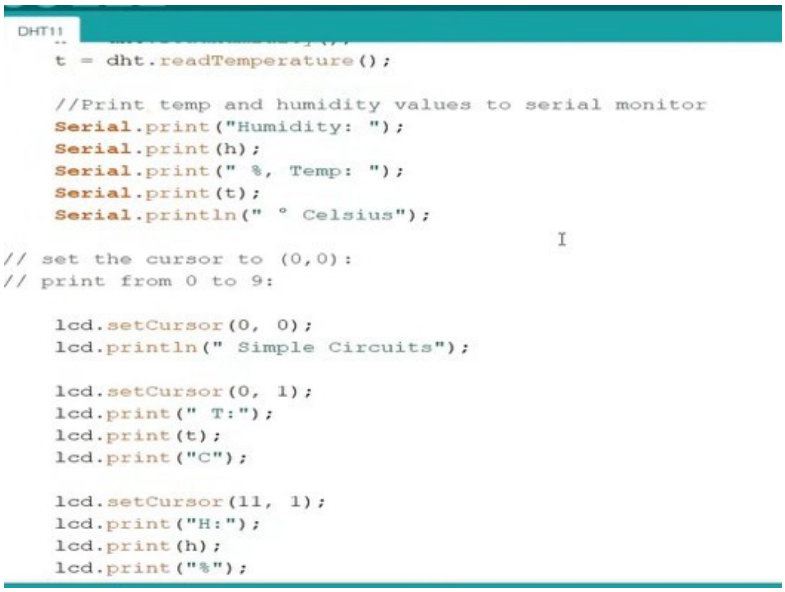
VCC 5V*
DATA Digital Pin 2

GND GND

Check the voltage requirements of your module. A bare DHT22 sensor may require a pull-up resistor on its data line; some modules already include one.

Procedure

1. Identify the DHT22 sensor and its pins.
2. Connect VCC to the appropriate power supply and GND to Arduino GND.
3. Connect the DATA pin to digital pin 2 of Arduino.
4. Connect Arduino UNO to the computer using a USB cable.
5. Open Arduino IDE and select Arduino UNO and the correct port.
6. Install the DHT sensor library using the Library Manager.
7. Upload the program given below.



```
DHT11
    t = dht.readTemperature();

    //Print temp and humidity values to serial monitor
    Serial.print("Humidity: ");
    Serial.print(h);
    Serial.print(" %, Temp: ");
    Serial.print(t);
    Serial.println(" ° Celsius");

// set the cursor to (0,0):
// print from 0 to 9:

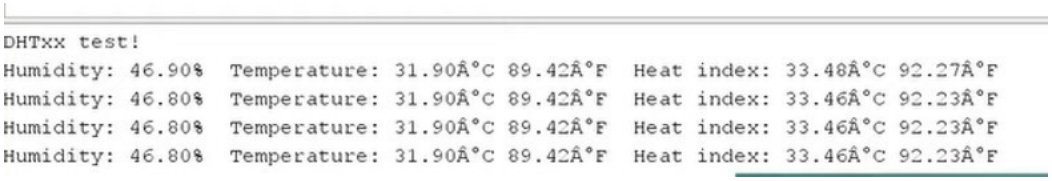
    lcd.setCursor(0, 0);
    lcd.println(" Simple Circuits");

    lcd.setCursor(0, 1);
    lcd.print(" T:");
    lcd.print(t);
    lcd.print("C");

    lcd.setCursor(11, 1);
    lcd.print("H:");
    lcd.print(h);
    lcd.print("%");
```

8. Open the Serial Monitor and set the baud rate to 9600.

9. Observe the temperature and humidity readings.



```
DHTxx test!
Humidity: 46.90%  Temperature: 31.90Â°C 89.42Â°F  Heat index: 33.48Â°C 92.27Â°F
Humidity: 46.80%  Temperature: 31.90Â°C 89.42Â°F  Heat index: 33.46Â°C 92.23Â°F
Humidity: 46.80%  Temperature: 31.90Â°C 89.42Â°F  Heat index: 33.46Â°C 92.23Â°F
Humidity: 46.80%  Temperature: 31.90Â°C 89.42Â°F  Heat index: 33.46Â°C 92.23Â°F
```

10. Record the readings under different environmental conditions.

Arduino Program

```
#include <DHT.h>

#define DHTPIN 2

#define DHTTYPE DHT22

DHT dht(DHTPIN,

DHTTYPE);

void setup() {

 Serial.begin(9600);

 dht.begin();

}

void loop() {

 float humidity = dht.readHumidity();

 float temperature =

 dht.readTemperature(); if

 (isnan(humidity) || isnan(temperature))

 { Serial.println("Sensor reading failed");
```

```
delay(2000);
return;
}
Serial.print("Temperature: ");
Serial.print(temperature);
Serial.println(" C");
Serial.print("Humidity: ");
Serial.print(humidity);
Serial.println(" %");
delay(2000);
}
```

Output

The temperature in degrees Celsius and relative humidity in percentage are displayed on the Serial Monitor.

Result

The DHT22 sensor was interfaced with Arduino UNO, and the measurement of temperature and relative humidity was studied.

Precautions

1. Check the sensor pin configuration before connecting.
2. Ensure proper power supply and common ground.
3. Install the required DHT library.
4. Avoid touching the sensing element during measurement.
5. Allow sufficient time between readings.

Experiment-10

To Interface GPS Module with Arduino and Display

Location Aim

To interface a GPS module with Arduino Uno and display the latitude and longitude of the current location on the Serial Monitor.

Apparatus Required

1. Arduino Uno
2. GPS Module
3. GPS Antenna
4. Laptop
5. USB Cable

6. Jumper Wires

Theory

GPS (Global Positioning System) is a satellite-based navigation system used to determine the geographical location of an object.

The GPS module receives signals from satellites and sends location data to Arduino through serial communication. The Arduino reads and processes this data to obtain latitude and longitude.

Circuit Connections

Assuming your GPS module uses UART serial communication:

GPS Module Arduino Uno

VCC Suitable power supply*

GND GND

TX Pin 4

RX Pin 3

Check the module's voltage rating before connecting VCC. GPS TX connects to Arduino RX, and GPS RX connects to Arduino TX.

Program

Install the TinyGPSPlus library from Arduino IDE → Library

Manager. #include <SoftwareSerial.h>

#include <TinyGPSPlus.h>

SoftwareSerial gpsSerial(4, 3);

TinyGPSPlus gps;

void setup() {

Serial.begin(9600);

gpsSerial.begin(9600);

Serial.println("GPS Module Started");

}

void loop() {

while (gpsSerial.available()) {

gps.encode(gpsSerial.read());

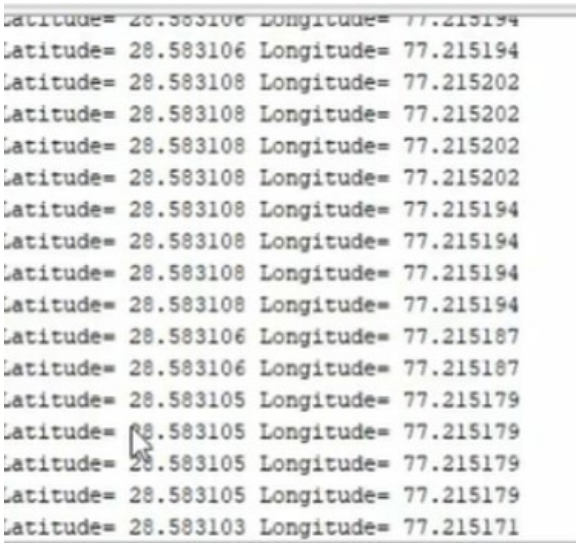
```
if (gps.location.isUpdated())
{ Serial.print("Latitude: ");
Serial.println(gps.location.lat(), 6);
Serial.print("Longitude: ");
Serial.println(gps.location.lng(),
6); Serial.println("------------");
}
}
}
```

Procedure

1. Connect the GPS module to Arduino using jumper wires as per the circuit connections.
2. Connect the GPS antenna to the module.
3. Connect Arduino to the laptop using the USB cable.
4. Open Arduino IDE and install the TinyGPSPlus library.
5. Select Arduino Uno and the correct COM port.
6. Upload the program.



7. Place the GPS antenna near an open sky for satellite reception.
8. Open Serial Monitor and set baud rate to 9600.
9. Observe the latitude and longitude readings.



Result

The GPS module was interfaced with Arduino Uno to obtain geographical location data.

Precautions

1. Check the GPS module's operating voltage before connecting it.
2. Connect TX and RX pins correctly.
3. Keep the GPS antenna in an open area for better satellite reception.
4. The first location fix may take several minutes.
5. If no readings appear, check the module's baud rate and wiring.